

Manging Migrations - EF Core - Exercise

Welcome to Managing Migrations with EF Core

In this exercise, you will learn how to create, manage, and apply migrations to a database using EF Core. This is an essential skill for any developer working with data-driven applications, as it allows you to keep your database schema in sync with your application's data model.

What You Will Learn

Database First Approach

We will **start with an existing database** and **generate the corresponding EF Core model classes**

Creating Migrations

You will learn **how to create migrations to update the database schema** when the model changes

Applying Migrations

We will cover **how to apply migrations to your database** to keep it up-to-date

Seeding Database

Learn how to **seed the database with initial data** during the migration process

Handling Errors

As with any development process, **you may encounter errors**. We will learn **how to troubleshoot** and **resolve common issues** that arise during migrations

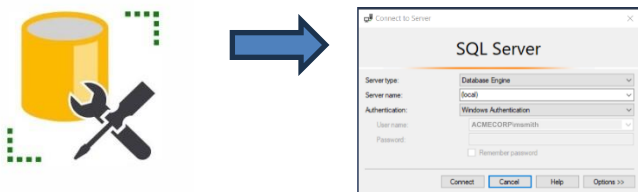
Practical Guide

1. Database First Approach

Save the Script

Save the provided SQL script as **migrations-dbset.sql**

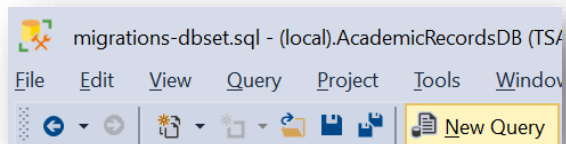
SQL Server Management Studio (SSMS)



Open SSMS and Connect to your SQL Server instance

Create a New Query

Open a **New Query** window



Copy and paste the contents of the **migrations-dbset.sql** script into the query window

```
CREATE DATABASE AcademicRecordsDB;
GO

USE AcademicRecordsDB;
GO

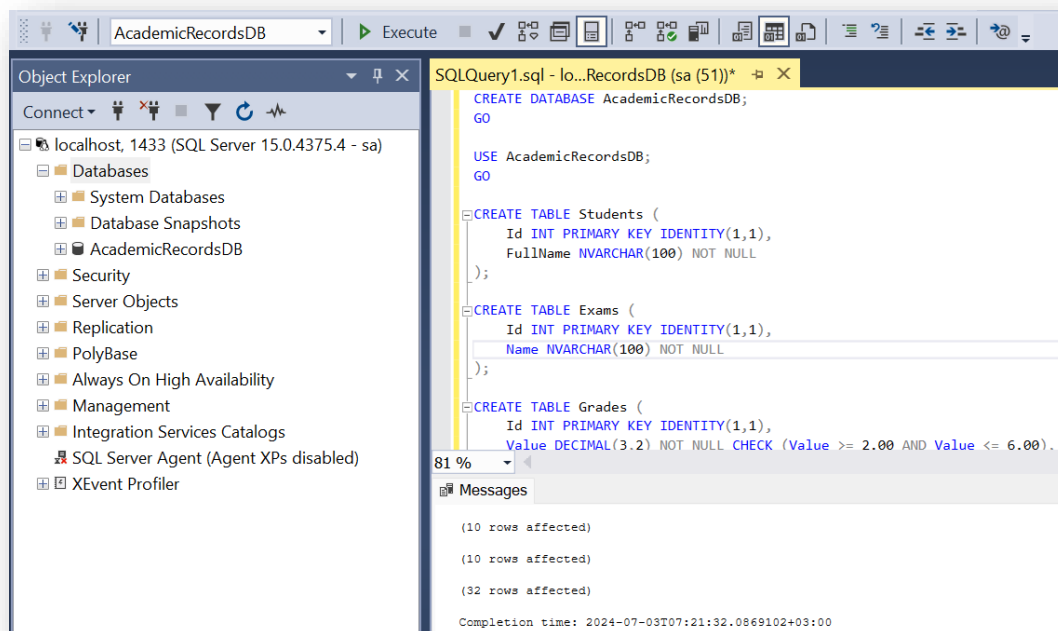
--CREATE TABLE Students (
    Id INT PRIMARY KEY IDENTITY(1,1),
    FullName NVARCHAR(100) NOT NULL
);

--CREATE TABLE Exams (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Name NVARCHAR(100) NOT NULL
);

--CREATE TABLE Grades (
    Id INT PRIMARY KEY IDENTITY(1,1),
    Value DECIMAL(3,2) NOT NULL CHECK (Value >= 2.00 AND Value <= 6.00),
    ExamId INT NOT NULL,
    StudentId INT NOT NULL,
    CONSTRAINT FK_Grades_Exams FOREIGN KEY (ExamId) REFERENCES Exams(Id),
    CONSTRAINT FK_Grades_Students FOREIGN KEY (StudentId) REFERENCES Students(Id)
);
```

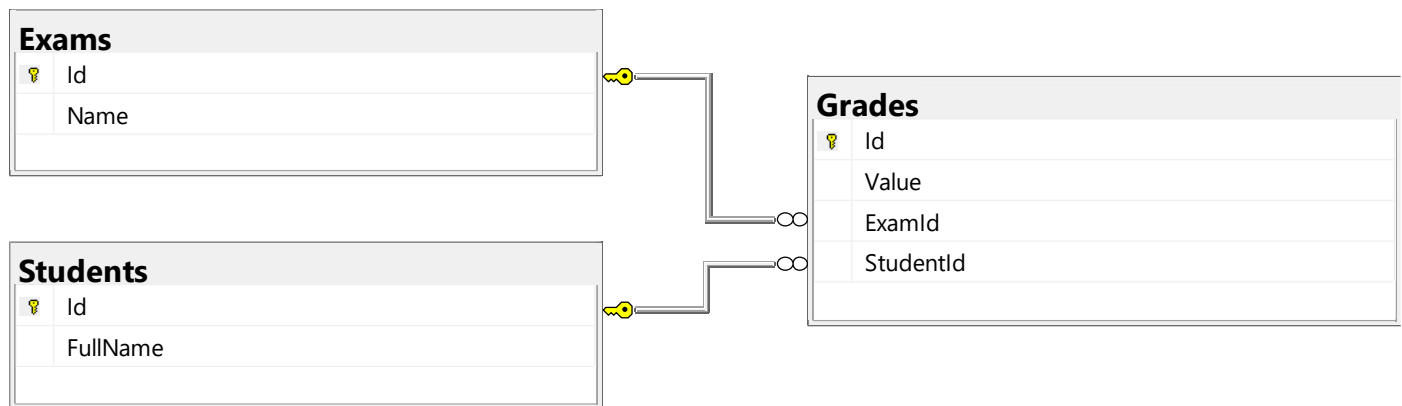
Execute the Script

Execute the script to create the **AcademicRecordsDB** database, the schema, and populate it with sample data



Simple Database Diagram

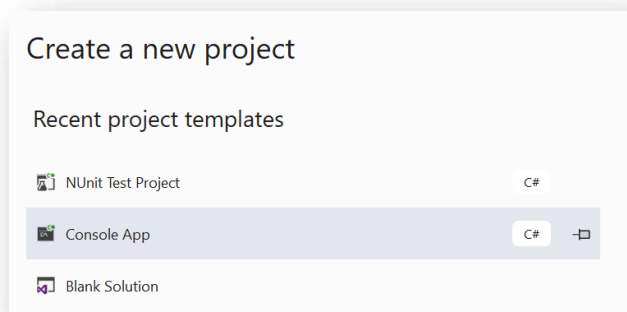
Now we have managed to create our **database in it's initial state**



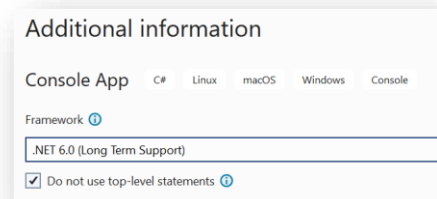
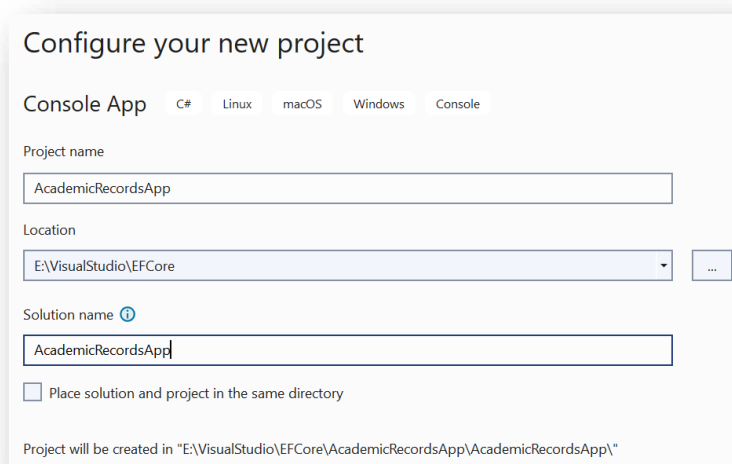
2. Creating a CS Project and Scaffolding Database

Set Up the Project in Visual Studio

Launch **Visual Studio** and select **Create a new project**

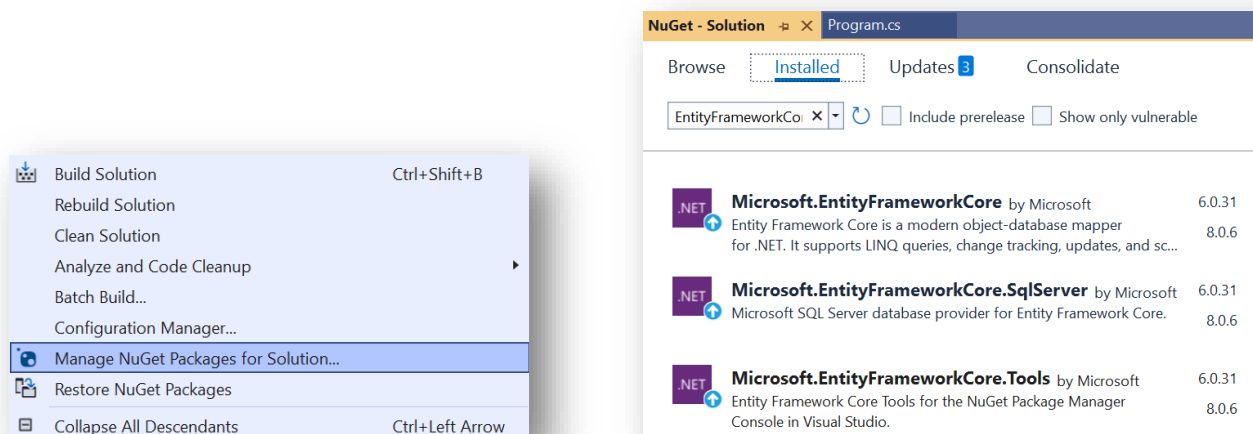


Create New **Console App** (.NET Core) -> Name your project (e.g., **AcademicRecordsApp**) and click **Create**



Add EF Core Packages

Right-click on the **project** in **Solution Explorer** and select **Manage NuGet Packages**



Run the Scaffold Command

In the **Package Manager Console**, run the following command to scaffold the database

If the **SQL Server instance is running on the local machine**, the server name might be **localhost** or **'.'**

```
Scaffold-DbContext "Server=YOUR_SERVER_NAME;Database=AcademicRecordsDB;Trusted_Connection=True;" Microsoft.EntityFrameworkCore.SqlServer -OutputDir Models
```

This command will **generate entity classes** and a **context class** based on the **existing database schema**

Verify the Scaffolded Code

After running the scaffold command, a **Models** folder will be **created (or updated)** in your project directory



This folder will contain the **entity classes** representing your database tables (**Students, Exams, Grades**) as well as the **context class** (**AcademicRecordsDbContext**)

Open the **generated entity classes** and the **context class** to **familiarize yourself with the structure and relationships** defined

The Process of Scaffolding

Scaffolding and **migrations** are both essential aspects of working with Entity Framework Core, but they serve different purposes and are used in different contexts

Scaffolding is the **process of generating code based on an existing database schema**. It is used to **create entity classes and a DbContext class that correspond to the tables and relationships** in your database

You **use scaffolding** when you have an **existing database** and you want to **generate the initial model code** to start working with EF Core in your application

Differences Between Scaffolding and Migrations

Starting Point:

- **Scaffolding:** Starts with an existing database schema and generates the corresponding model code
- **Migrations:** Starts with an existing model and generates code to update the database schema

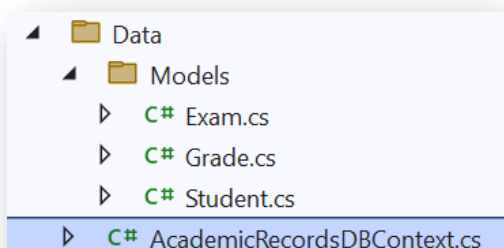
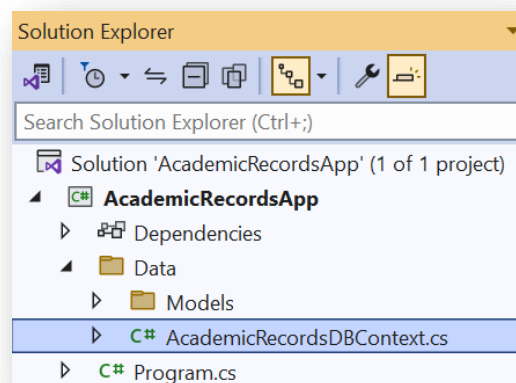
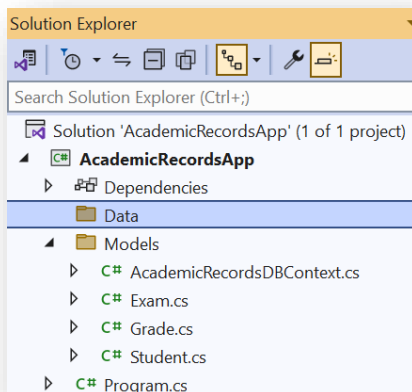
Purpose:

- **Scaffolding:** Database-first approach (from database to code)
- **Migrations:** Code-first or model-first approach (from code to database)

Why Scaffolding is Not a Migration:

- **Scaffolding** is a **one-time process** to generate the initial code based on an existing database. It **does not track changes or apply incremental updates**
- **Migrations** are used to **manage and apply incremental updates** to the database schema over time. They **track changes made to the model and generate scripts to apply those changes to the database**

Organize the Project Structure and Rebuild the Solution



3. Detailed Analysis of AcademicRecordsDBContext

Namespace and Using Directives

The namespace is **AcademicRecordsApp.Data**, which aligns with the new folder structure

It includes **necessary directives for EF Core** and collections

```
using AcademicRecordsApp.Data.Models;
using Microsoft.EntityFrameworkCore;

namespace AcademicRecordsApp.Data
{
}
```

Constructors

The **parameterless constructor** is useful for scenarios where options are set externally

The constructor with **DbContextOptions<AcademicRecordsDBContext>** allows passing configuration options when instantiating the context

```
public partial class AcademicRecordsDBContext : DbContext
{
    0 references
    public AcademicRecordsDBContext()
    {
    }

    0 references
    public AcademicRecordsDBContext(DbContextOptions<AcademicRecordsDBContext> options)
        : base(options)
    {
    }
}
```

DbSet Properties

```
0 references
public virtual DbSet<Exam> Exams { get; set; } = null!;
0 references
public virtual DbSet<Grade> Grades { get; set; } = null!;
0 references
public virtual DbSet<Student> Students { get; set; } = null!;
```

These properties **represent the tables in the database** and **provide a way to query and save instances** of these entities

OnConfiguring Method

This method is **called during context configuration**, it **sets up the connection string for the SQL Server database**.

```

0 references
protected override void OnConfiguring(DbContextOptionsBuilder optionsBuilder)
{
    if (!optionsBuilder.IsConfigured)
    {
        // To protect potentially sensitive information in your connection string, you should
        // avoid scaffolding the connection string by using the Name= syntax to read it from
        // the configuration. For more guidance on storing connection strings, see
        // https://aka.ms/aspnetcoreConnectionString.
        optionsBuilder.UseSqlServer("YourConnectionStringHere");
    }
}

```

The **#warning** directive suggests moving the connection string out of the source code for security reasons

OnModelCreating Method

Configures the model's properties and relationships using the Fluent API

Defines constraints and relationships for the **Exam**, **Grade**, and **Student** entities

```

0 references
protected override void OnModelCreating(ModelBuilder modelBuilder)
{
    modelBuilder.Entity<Exam>(entity =>
    {
        entity.Property(e => e.Name).HasMaxLength(100);
    });

    modelBuilder.Entity<Grade>(entity =>
    {
        entity.Property(e => e.Value).HasColumnType("decimal(3, 2)");

        entity.HasOne(d => d.Exam)
            .WithMany(p => p.Grades)
            .HasForeignKey(d => d.ExamId)
            .OnDelete(DeleteBehavior.ClientSetNull)
            .HasConstraintName("FK_Grades_Exams");

        entity.HasOne(d => d.Student)
            .WithMany(p => p.Grades)
            .HasForeignKey(d => d.StudentId)
            .OnDelete(DeleteBehavior.ClientSetNull)
            .HasConstraintName("FK_Grades_Students");
    });

    modelBuilder.Entity<Student>(entity =>
    {
        entity.Property(e => e.FullName).HasMaxLength(100);
    });

    OnModelCreatingPartial(modelBuilder);
}

```

Partial Method

```

1 reference
partial void OnModelCreatingPartial(ModelBuilder modelBuilder);

```

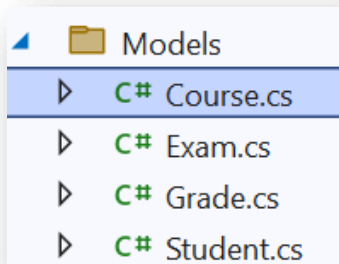
This partial method allows for further customization in another partial class

4. Evolving Database and Models

This section will guide you through the process of evolving your database schema and models. You will learn how to add new entities, modify existing ones, and apply these changes to the database using EF Core migrations

Adding a New Entity: Course

Add a new class called `Course.cs` in the Models folder



```
namespace AcademicRecordsApp.Data.Models
{
    0 references
    public class Course
    {
        0 references
        public int Id { get; set; }
        0 references
        public string Name { get; set; } = string.Empty;
        0 references
        public ICollection<Student> Students { get; set; } = new List<Student>();
    }
}
```

Update the Student entity class. Modify the Student class to include a collection of Courses

```
public partial class Student
{
    0 references
    public Student()
    {
        Grades = new HashSet<Grade>();
    }

    0 references
    public int Id { get; set; }
    1 reference
    public string FullName { get; set; } = null!;

    2 references
    public virtual ICollection<Grade> Grades { get; set; }

    0 references
    public ICollection<Course> Courses { get; set; } = new List<Course>();
}
```

Update the DbContext

Add a DbSet for the Course entity in `AcademicRecordsDbContext.cs`

```
0 references
public virtual DbSet<Exam> Exams { get; set; } = null!;
0 references
public virtual DbSet<Grade> Grades { get; set; } = null!;
0 references
public virtual DbSet<Student> Students { get; set; } = null!;

0 references
public virtual DbSet<Course> Courses { get; set; } = null!;
```



```
modelBuilder.Entity<Course>(entity =>
{
    entity.Property(e => e.Name).HasMaxLength(100);
});
```

Set the **maximum length** for the Name property to **100 characters**. **HasMaxLength()** is a method that specifies the maximum length for a string property in the database

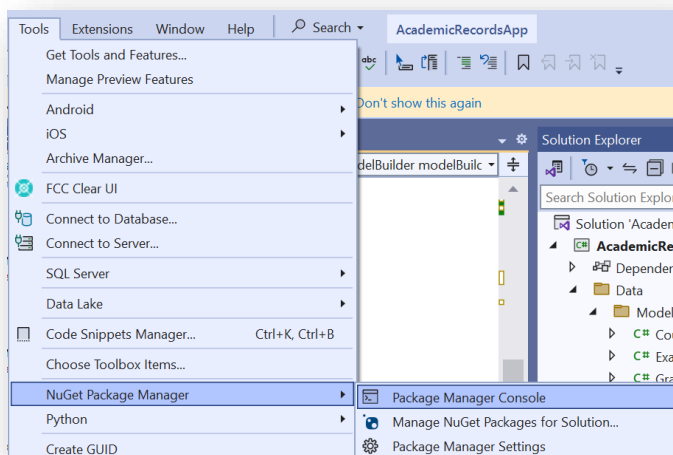
Creating and Applying the Migration

Ensure the Model Changes:

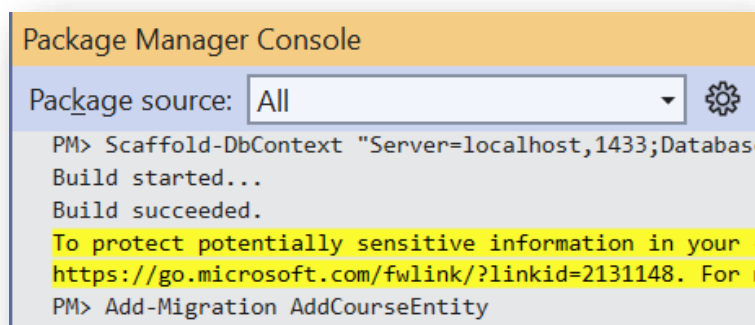
- Created **Course.cs**
- Updated **Student.cs**
- Updated **AcademicRecordsDbContext.cs**

Add a Migration

- Open the **Package Manager Console** (Go to Tools > NuGet Package Manager > Package Manager Console)



- Run the **Add-Migration** Command



- **Review the Migration Script**

Visual Studio will **generate a migration script**. Review the script to understand what changes will be applied to the database

- 📁 Migrations
 - 📄 C# 20240703072747_AddCourseEntity.cs
 - 📄 C# AcademicRecordsDbContextModelSnapshot.cs

```
public partial class AddCourseEntity : Migration
{
    0 references
    protected override void Up(MigrationBuilder migrationBuilder)
    {
        migrationBuilder.CreateTable(
            name: "Courses",
            columns: table => new
            {
                Id = table.Column<int>(type: "int", nullable: false)
                    .Annotation("SqlServer:Identity", "1, 1"),
                Name = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable: false)
            },
            constraints: table =>
            {
                table.PrimaryKey("PK_Courses", x => x.Id);
            });
    }
}
```

Update-Database

Run the Update-Database Command in the Package Manager Console

Package Manager Console

Package source: All

```
PM> Scaffold-DbContext "Server=localhost,1433;Data Source=AcademicRecords;Initial Catalog=AcademicRecords;Trusted_Connection=True;" Microsoft.EntityFrameworkCore.SqlServer
Build started...
Build succeeded.
To protect potentially sensitive information in your application, we recommend adding the 'appsettings.json' file to the 'bin' folder and run the command 'dotnet user-secrets init'.
For more information, see https://go.microsoft.com/fwlink/?linkid=2131148.
PM> Add-Migration AddCourseEntity
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
```

Encounter Update-Database Errors

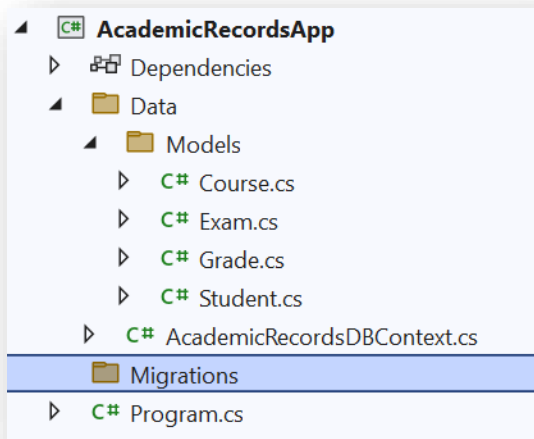
```
Applying migration '20240703080144_AddCourseEntity'.
Failed executing DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
CREATE TABLE [Exams] (
    [Id] int NOT NULL IDENTITY,
    [Name] nvarchar(100) NOT NULL,
    CONSTRAINT [PK_Exams] PRIMARY KEY ([Id])
);
```

The migration is **creating the Exams, Students, and other tables** because it **doesn't have any record of the current database schema**. This can happen if the **migration history is out of sync with the actual database schema**, or if the **initial migration didn't accurately reflect the existing state of the database**.

Sync Migration History with the Database

Remove Existing Migrations

- Delete all existing migration files in the Migrations folder



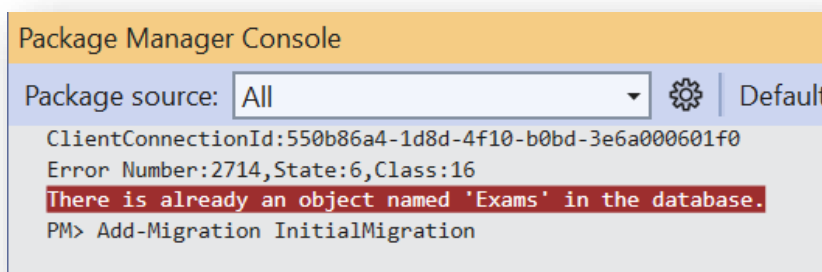
Reset the Migration History

- If necessary, you can manually reset the migration history table in the database. This is typically the `__EFMigrationsHistory` table
- Open SQL Server Management Studio (SSMS) and run the following command to clear the migration history

```
USE AcademicRecordsDB;  
GO  
  
DELETE FROM __EFMigrationsHistory;
```

Create an Initial Baseline Migration

- Create a new initial migration that reflects the current state of the database without trying to create tables that already exist



Manually Adjust the Initial Migration

- Open the generated migration file and **remove or comment out the creation of existing tables**

```
protected override void Up(MigrationBuilder migrationBuilder)
{
    migrationBuilder.CreateTable(
        name: "Courses",
        columns: table => new
        {
            Id = table.Column<int>(type: "int", nullable: false)
                .Annotation("SqlServer:Identity", "1, 1"),
            Name = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable: false)
        },
        constraints: table =>
        {
            table.PrimaryKey("PK_Courses", x => x.Id);
        });

    //migrationBuilder.CreateTable(
    //    name: "Exams",
    //    columns: table => new
    //    {
    //        Id = table.Column<int>(type: "int", nullable: false)
    //            .Annotation("SqlServer:Identity", "1, 1"),
    //        Name = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable: false)
    //    },
    //    constraints: table =>
    //    {
    //        table.PrimaryKey("PK_Exams", x => x.Id);
    //    });
}
```

```
//migrationBuilder.CreateTable(
//    name: "Students",
//    columns: table => new
//    {
//        Id = table.Column<int>(type: "int", nullable: false)
//            .Annotation("SqlServer:Identity", "1, 1"),
//        FullName = table.Column<string>(type: "nvarchar(100)", maxLength: 100, nullable: false)
//    },
//    constraints: table =>
//    {
//        table.PrimaryKey("PK_Students", x => x.Id);
//    });
}
```

```
//migrationBuilder.CreateTable(
//    name: "Grades",
//    columns: table => new
//    {
//        Id = table.Column<int>(type: "int", nullable: false)
//            .Annotation("SqlServer:Identity", "1, 1"),
//        Value = table.Column<decimal>(type: "decimal(3,2)", nullable: false),
//        ExamId = table.Column<int>(type: "int", nullable: false),
//        StudentId = table.Column<int>(type: "int", nullable: false)
//    },
//    constraints: table =>
//    {
//        table.PrimaryKey("PK_Grades", x => x.Id);
//        table.ForeignKey(
//            name: "FK_Grades_Exams",
//            column: x => x.ExamId,
//            principalTable: "Exams",
//            principalColumn: "Id");
//        table.ForeignKey(
//            name: "FK_Grades_Students",
//            column: x => x.StudentId,
//            principalTable: "Students",
//            principalColumn: "Id");
//    });
}
```

Ensure that the Down method does not drop the existing tables (Exams, Students, Grades). You should only drop the tables that were added in this migration (Courses and CourseStudent).

```
protected override void Down(MigrationBuilder migrationBuilder)
{
    migrationBuilder.DropTable(
        name: "CourseStudent");

    //migrationBuilder.DropTable(
    //    name: "Grades");

    migrationBuilder.DropTable(
        name: "Courses");

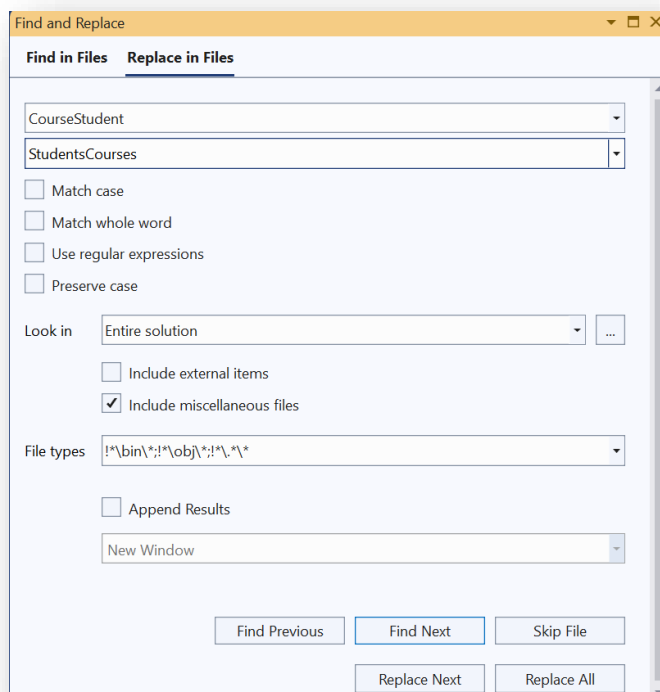
    //migrationBuilder.DropTable(
    //    name: "Exams");

    //migrationBuilder.DropTable(
    //    name: "Students");
}
```

Rename Using Search and Replace

Yes, it is generally better to have the join table named **StudentsCourses** instead of **CourseStudent**, following the **convention of placing the primary entity name first**, which helps in **readability and consistency**.

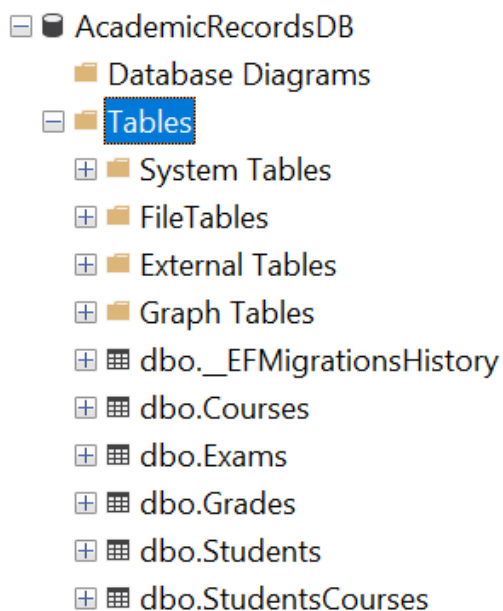
- Press **Ctrl + Shift + H** to open the **"Replace in Files"** dialog



```
migrationBuilder.CreateTable(
    name: "StudentsCourses",
    columns: table => new
```

Update Database

```
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240703081759_InitialMigration'.
Done.
PM>
```



Error Explanation

Initial Error:

- We **created the Course entity** and **updated the Student entity** to include a collection of Courses **without properly configuring the relationships** in the **OnModelCreating** method
- As a result, **the generated migration script attempted to recreate existing tables** (Exams, Students, Grades), leading to conflicts and errors

Lack of Fluent API Configuration:

- **Without using the Fluent API to configure the many-to-many relationship**, EF Core was **unable to correctly create and manage the join table** for the Students and Courses relationship
- This **caused issues with the migration process** and could lead to an inconsistent or incorrect database schema
- The **Fluent API allows** you to explicitly **define the relationships between entities**, ensuring that EF Core correctly understands and manages the relationships
- **Proper configuration helps avoid conflicts and errors during the migration process**, ensuring that existing tables and data are not inadvertently modified or deleted

Correct Configuration with Fluent API:

- To correct the mistake, the OnModelCreating method should be updated in the **AcademicRecordsDbContext** to explicitly define the **many-to-many relationship** between **Students** and **Courses** using the **Fluent API**

```
modelBuilder.Entity<Student>()
    .HasMany(s => s.Courses)
    .WithMany(c => c.Students)
    .UsingEntity<Dictionary<string, object>>(
        "StudentsCourses",
        j => j.HasOne<Course>().WithMany().HasForeignKey("CoursesId"),
        j => j.HasOne<Student>().WithMany().HasForeignKey("StudentsId"),
        j =>
        {
            j.HasKey("StudentsId", "CoursesId");
            j.ToTable("StudentsCourses");
        }
    );
```

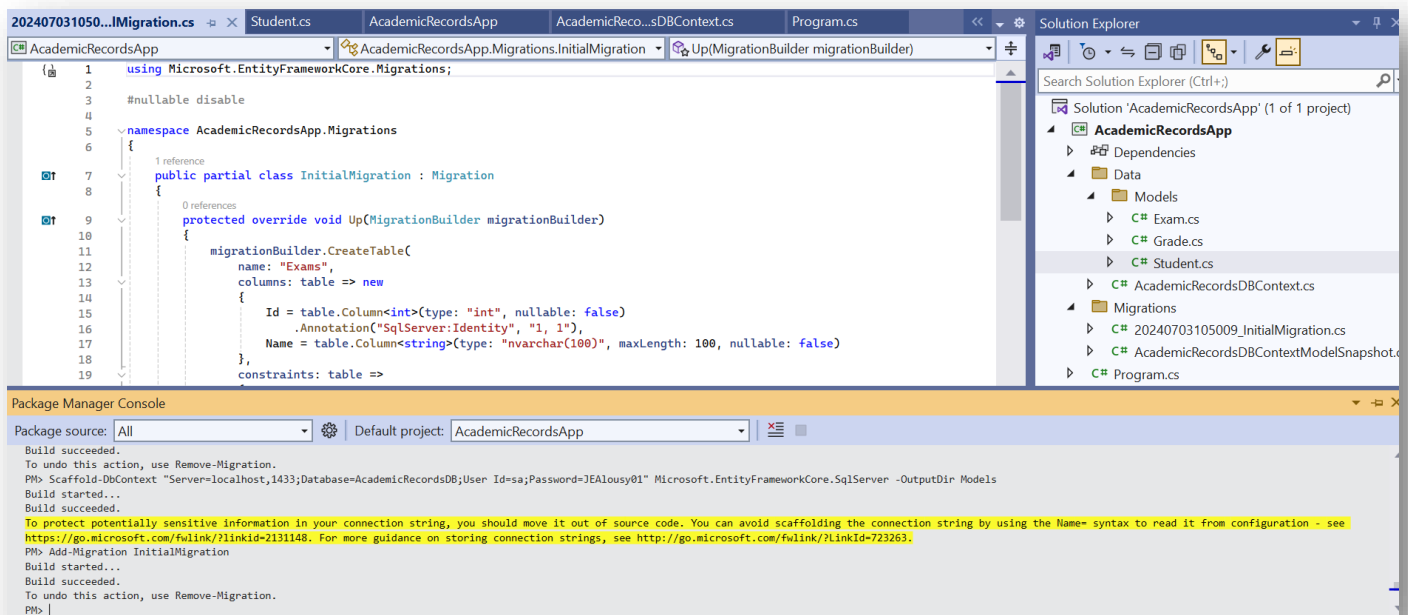
Resolve the Issue

Remove Existing Migrations

- Delete all existing migration files and reset the migration history in the database to ensure a clean state

Create an Initial Baseline Migration

- Let's start from the beginning by **scaffolding the database again** and then **creating an initial migration**



Creating Course Entity & Configure Relationships Using Fluent API



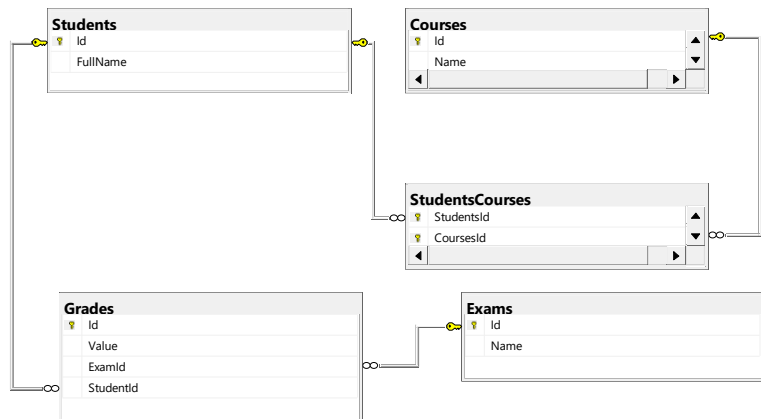
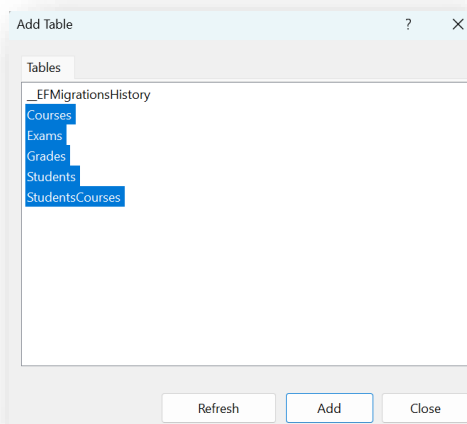
MigrationFile-AddCourse.txt

Apply the Migration

```
PM> Add-Migration AddCourseEntity
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240703111906_AddCourseEntity'.
Done.
PM>
```

Updated Database Diagram

Refresh the existing tables, and **build a New Diagram**



In the **updated database schema**, we have introduced **two significant changes**. First, **we added a new table** called **Courses** to store information about the various courses offered, each identified by a unique **Id** and a **Name**. Second, we established a **many-to-many relationship between Students and Courses** using a join table named **StudentsCourses**. This join table includes foreign keys **StudentsId** and **CoursesId**, linking students to the courses they are enrolled in.

5. Creating New Data Model Relations

In this section, we will focus on **establishing a new relationship between Student, Course and Exam entities**. Specifically, we will define that **every exam belongs to one course only**, while **one course can have many exams**. This relationship is known as a **one-to-many relationship**. Additionally, we will establish a **many-to-many relationship between Student and Course**, where **every student can enroll in multiple courses**, and **each course can have many students**. Follow these steps to create and configure these relationships in your data model.

Using Data Annotations to Configure Relationships

Data Annotations provide a **way to configure the model classes directly using attributes**. This can make the model classes more readable by keeping the configuration close to the properties they configure.

Update Entities

Exam Entity:

- In the Exam entity, we have introduced several enhancements **using Data Annotations** to ensure proper **configuration and validation of the database schema**.
- We added a **CourseId** property to **serve as a foreign key**, which establishes a **one-to-many relationship** with the **Course** entity. This is enforced by the `[ForeignKey("CourseId")]` annotation.
- Additionally, we used the `[Required]` and `[MaxLength(100)]` annotations on the Name property to ensure that every exam has a name and that it does not exceed 100 characters in length.
- The Course **navigation property** provides a **link to the associated Course entity**, allowing us to navigate from an Exam to the Course it belongs to.
- These changes ensure that each exam is associated with a specific course, enhancing data integrity and the logical structure of our database.



ExamEntity.txt

```
public class Exam
{
    0 references
    public Exam()
    {
        Grades = new HashSet<Grade>();
    }

    [Key]
    0 references
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    1 reference
    public string Name { get; set; } = null!;

    // Foreign key property
    [Required]
    [ForeignKey("Course")]
    0 references
    public int CourseId { get; set; }

    // Navigation property
    0 references
    public virtual Course Course { get; set; } = null!;

    2 references
    public virtual ICollection<Grade> Grades { get; set; }
}
```

Course Entity:

- In the **Course** entity, we have implemented several enhancements **using Data Annotations** to ensure **accurate configuration and validation of the database schema**
- We introduced a **Name** property with the `[Required]` and `[MaxLength(100)]` annotations to guarantee that each course has a name and that the name does not exceed 100 characters in length
- The **Exams navigation property** was added to represent the collection of Exam entities associated with a particular course. This creates a **one-to-many relationship**, allowing each course to have multiple exams
- These changes enhance the relational structure of the database, ensuring that courses can be linked to their respective exams, thereby improving data organization and integrity. The Course entity now accurately reflects the academic structure, where each course can encompass various exams



CourseEntity.txt

```
public class Course
{
    10 references
    public Course()
    {
        Exams = new HashSet<Exam>();
        Students = new HashSet<Student>();
    }

    [Key]
    10 references
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    11 references
    public string Name { get; set; } = null!;

    // Navigation properties
    2 references
    public virtual ICollection<Exam> Exams { get; set; }
    2 references
    public virtual ICollection<Student> Students { get; set; }
}
```

Student Entity:

- In the Student entity, we have introduced several enhancements **using Data Annotations** to ensure proper **configuration and validation of the database schema**
- We added a FullName property with the **[Required]** and **[MaxLength(100)]** annotations to ensure that **every student has a full name and that it does not exceed 100 characters in length**
- The **Grades navigation property** was added to represent the collection of Grade entities associated with a particular student. This allows **each student to have multiple grades for different exams**
- The **Courses navigation property** was added to establish a **many-to-many relationship between Student and Course entities**. This allows **each student to enroll in multiple courses**, and **each course to have multiple students**
- These changes enhance the relational structure of the database, ensuring that students can be linked to their respective grades and courses, thereby improving data organization and integrity. The Student entity now accurately reflects the academic structure, where each student can have multiple grades and enroll in multiple courses



StudentEntity.txt

```

public class Student
{
    0 references
    public Student()
    {
        Grades = new HashSet<Grade>();
        Courses = new HashSet<Course>();
    }

    [Key]
    0 references
    public int Id { get; set; }

    [Required]
    [MaxLength(100)]
    1 reference
    public string FullName { get; set; } = null!;

    // Navigation properties
    2 references
    public virtual ICollection<Grade> Grades { get; set; }
    2 references
    public virtual ICollection<Course> Courses { get; set; }
}

```

Update DbContext

In this section, we will **configure the relationships and constraints in our DbContext using the Fluent API**. This configuration ensures that our entity relationships are accurately represented in the database schema.

Configuring Relationships in DbContext

We will update the **OnModelCreating** method in the **DbContext** to **define the relationships** between the **Student**, **Course**, and **Exam** entities **using the Fluent API**



ContextUpdated.txt

Add the Migration and Update the Database

```

PM> Add-Migration StudentExamCourse_RelationsAdded
Build started...
Build succeeded.
To undo this action, use Remove-Migration.

```

- We have successfully added the new migration called **StudentExamCourse_RelationsAdded**
- We are going to attempt to **update the database with this new migration**

```
Package Manager Console
Package source: All [v] [g] Default project: AcademicRecordsApp [v] [x] [ ]
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240703144003_StudentExamCourse_RelationsAdded'.
Failed executing DbCommand (16ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
ALTER TABLE [Exams] ADD CONSTRAINT [FK_Exams_Courses_CourseId] FOREIGN KEY ([CourseId]) REFERENCES [Courses] ([Id]) ON DELETE CASCADE;
Microsoft.Data.SqlClient.SqlException (0x80131904): The ALTER TABLE statement conflicted with the FOREIGN KEY constraint "FK_Exams_Courses_CourseId". The conflict occurred in database "AcademicRecords", table "Exams", column 'Id'.
    at Microsoft.Data.SqlClient.SqlConnection.OnError(SqlException exception, Boolean breakConnection, Action`1 wrapCloseInAction)
    at Microsoft.Data.SqlClient.SqlInternalConnection.OnError(SqlException exception, Boolean breakConnection, Action`1 wrapCloseInAction)
    at Microsoft.Data.SqlClient.TdsParser.ThrowExceptionAndWarning(TdsParserStateObject stateObj, Boolean callerHasConnectionLock, Boolean asyncClose)
    at Microsoft.Data.SqlClient.TdsParser.TryRun(RunBehavior runBehavior, SqlCommand cmdHandler, SqlDataReader dataStream, BulkCopySimpleResultSet bulkCopyResultSet, Int32 userTimeout, Int32 cbTimeout)
```

- We attempted to update the database with the new migration, but encountered an error
- The error message indicates that **there was a conflict with the FOREIGN KEY constraint**

Understanding the Issue

The error occurs because the **Exams table has a foreign key constraint that references the Courses table**. The specific issue is likely **caused by existing data in the Exams table that doesn't match any Id in the Courses table**

This can happen if:

- There are records in the Exams table with CourseId values that don't exist in the Courses table
- The CourseId column in the Exams table has NULL values, and the foreign key constraint doesn't allow NULLs

Resolve the Problem

Remove the Failed Migration:

- Since the **update was unsuccessful**, we'll first **remove the failed migration**

```
PM> Remove-Migration
Build started...
Build succeeded.
Removing migration '20240703144003_StudentExamCourse_RelationsAdded'.
Reverting the model snapshot.
Done.
PM> |
```

- **Modify OnModelCreating to Allow Nulls Temporarily**

```
// Temporarily allow NULLs in CourseId
modelBuilder.Entity<Exam>(entity =>
{
    entity.Property(e => e.CourseId).IsRequired(false);
    entity.Property(e => e.Name).HasMaxLength(100);

    entity.HasOne(e => e.Course)
        .WithMany(c => c.Exams)
        .HasForeignKey(e => e.CourseId)
        .OnDelete(DeleteBehavior.Cascade);
});
```

What Will Happen Later When the Property is Made Required?

- **Schema Update**
 - EF Core will generate a migration that updates the schema to enforce the property as required (non-nullable)
 - This will alter the table definition to disallow nulls in the column
- **Data Validation**
 - Existing data must comply with the new non-nullable constraint. If there are any null values in the column, the migration will fail
 - EF Core will ensure at the application level that any new records or updates to this column must provide a non-null value
- **Foreign Key Constraints**
 - The foreign key relationship will enforce that every entry in the Exams table must reference a valid Id in the Courses table
 - This ensures data integrity by preventing orphaned records in the Exams table

Add and Apply a Migration

It seems that the migration will be successful this time

```
Package Manager Console
Package source: All
Build started...
Build succeeded.
Removing migration '20240703144003_StudentExamCourse_RelationsAdded'.
Reverting the model snapshot.
Done.
PM> Add-Migration StudentExamCourse_RelationsAdded_AllowNullsInCourseId
Build started...
Build succeeded.
```

But encounter a new issue

```
System.InvalidOperationException: The property 'Exam.CourseId' cannot be marked as nullable/optional because the type of the property is 'int' which is not a nullable type. Any property can be marked as non-nullable/required, but only properties of nullable types can be marked as nullable/optional.
at Microsoft.EntityFrameworkCore.Metadata.Internal.Property.SetIsNullable(Nullable`1 nullable, ConfigurationSource configurationSource)
at Microsoft.EntityFrameworkCore.Metadata.Internal.InternalPropertyBuilder.IsRequired(Nullable`1 required, ConfigurationSource configurationSource)
at Microsoft.EntityFrameworkCore.Metadata.Builders.PropertyBuilder.IsRequired(Boolean required)
at Microsoft.EntityFrameworkCore.Metadata.Builders.PropertyBuilder`1.IsRequired(Boolean required)
at AcademicRecordsApp.Data.AcademicRecordsDBContext.<>c.<OnModelCreating>b__19_0(EntityTypeBuilder`1 entity) in E:\VisualStudio\EFCore\AcademicRecordsApp\AcademicRecordsApp\Data\AcademicRecordsDBContext.cs:line 35
```

```
The property 'Exam.CourseId' cannot be marked as nullable/optional because the type of the property is 'int' which is not a nullable type. Any property can be marked as non-nullable/required, but only properties of nullable types can be marked as nullable/optional.
PM>
```

We encountered an **error while attempting to make the CourseId property nullable**. The error message indicates that **the property cannot be marked as nullable unless it is defined as a nullable type in the model class**

Nullable Integer Property Resolution

Remove the Failed Migration

- Remove the failed migration to revert the model snapshot

Update the Model Class

- Remove the Required Attribute
- Modify the Exam model to make the **CourseId** property a **nullable integer (int?)**

```
[ForeignKey("Course")]
2 references
public int? CourseId { get; set; }
```

Add a New Migration and Update the Database

```
PM> Remove-Migration
Build started...
Build succeeded.
Removing migration '20240703152259_StudentExamCourse_RelationsAdded_AllowNullsInCourseId'.
Reverting the model snapshot.
Done.
PM> Add-Migration StudentExamCourse_RelationsAdded_AllowNullsInCourseId
Build started...
Build succeeded.
To undo this action, use Remove-Migration.
PM> Update-Database
Build started...
Build succeeded.
Applying migration '20240703154421_StudentExamCourse_RelationsAdded_AllowNullsInCourseId'.
Done.
```

Completing the Migration and Updating the Database Schema

Congratulations on successfully creating the migration and updating the database schema!

